NYC Mobility Dashboard Report

Team: Devnote

Members

- Rene Guido Kayigamba
- Elijah Kabatsi
- Nyirihirwe Yves

1. Problem Framing and Dataset Analysis

1.1 Project Background

Daily large volumes of data are generated by the urban transportation systems. Businesses, transportation companies, and city planners are continuously trying to make better choices about infrastructure and mobility by having a better understanding of travel trends.

To review and show cab transportation statistics from New York City hence, the NYC Mobility Dashboard was made. Taxi trip records, which does include details like pickup and drop-off locations, the travel distance, the fare amount, the passenger count, and trip time, that are currently being used in the project.

The intended achievement was to actually use an interactive dashboard to really change the unprocessed transportation data into insightful visual representations.

1.2 Dataset Description

The dataset used was the NYC Taxi Trip Records dataset (yellow_tripdata_2025-01.parquet).

The crucial fields are:

- Pickup Location
- Dropoff Location
- Trip Distance
- Fare Amount
- Passenger Count
- Trip Duration

These unique properties give crucial insights within transportation demand and mobility trends.

1.3 Data Challenges

Quite a number of obstacles that were actually found while we were preparing the data:

Missing Values

Particularly in the trip distance and the passenger count columns, we found that they were some records that had missing data.

Outliers

There were certain excursions with exceptionally long distances or rates that very muchly differed greatly from normal the taxi rides.

Data Volume

The dataset really needed effective and efficient database import and query processing, this is because of the large amount of records.

Inconsistent Records

A couple of entries had values that were implausible, such short with very much high prices.

1.4 Data Cleaning Assumptions

These are the assumptions that we applied:

- Records with critical missing values were deleted in mysql.
- Negative fare values were also removed as it doesn’ t make sense.
- Trips with zero distance were actually kept only if other fields were filled properly.
- While we were reviewing, the really large values were considered possible outliers..

1.5 Unexpected Observation

A little rate of distances had really big prices compared to their reported distances, according to the research we made.

Rather than depending only on averages, which can be easily lead to extreme results, this insight encouraged a way of using summaries and trend visualizations in our dashboard design.

2. System Architecture and Design Decisions

2.1 System Architecture

Frontend (React + Vite)
        |
        v
Backend API (Flask)
        |
        v
MySQL Database
        |
        v
NYC Taxi Dataset

2.2 Technology Stack Selection

React

React was actually chosen as it is quite efficient and help in rendering of interactive user interfaces and allows component-based development. And we also are trying to get more experience with it's usage as this is the time for it.

Flask

Flask is easy to use for creating REST APIs hence, why we chose it.

MySQL

MySQL was chosen for its ability to store data reliably in a relational format that is efficient to query.

Chart.js

To create simple, yet interactive visualizations, Chart.js was used.

2.3 Database Design

Taxi trip data is very structured and can be easily aggregated using SQL, so a relational schema was chosen.

The schema supports:

- Quick retrieval of trip records.

- Statistical aggregation
- Dashboard reporting

2.4 Design Trade-offs

Simplicity vs Scalability

An architecture based on Flask was selected as an alternative to a microservices approach due to the reduced complexity of development.

Dashboard Performance vs Data Detail

To enhance performance the aggregation was done in the backend instead of loading all records into the frontend.

Advanced features or Development Speed.

The team focused on the most essential analytics features first, and added the more complex features like authentication and mapping later.

3. Algorithmic Logic and Data Structures

3.1 Custom Algorithm: Manual Trip Frequency Ranking

Since we didn't use any pre-built counting libraries, we decided to build our own frequency-counting algorithm to discover the most common pick up locations.

Problem

Identify the most common pick up locations in the data set.

Approach

A dictionary like structure is made up by hand.

For each trip:

1. Read pickup location.
2. Check if location exists.
3. Increase the number of it if there is.
4. If not, make another entry.
5. Count up to find the most common frequencies.

Pseudocode

Create empty frequency table.

FOR each trip

location = pickup_location

IF location exists

    count = count + 1

ELSE

    count = 1

ENDIF

ENDFOR

FOR each location

find highest count

ENDFOR

Return ranked locations

Time Complexity

O(n)

where n is the number of trips.

Space Complexity

O(k)

where k is the number of different pick-up locations.

Justification

This algorithm is efficient in performing frequency analysis, and it shows that the student has an understanding of how to use basic data structures and how to count, but without the use of specialized libraries.

4. Insights and Interpretation

Insight 1: Trip Volume Distribution

Method

Found the number of trips for each pickup location and plotted it as a bar graph.

Finding

High percentage of trips were from a small percentage of pickup locations.

Interpretation

This indicates that transportation demand is not evenly distributed throughout the city; it is mostly clustered in certain areas where commercial and business activity can be found.

Visual

(Embed a dashboard image with the distribution of pickup locations)

Insight 2: Changing of fare amounts.

Method

Derived the average fares and visualized them on the dash.

Finding

The price of fares was not uniform for each trip.

Interpretation

Fares vary between different routes because of variations in distances, time and traffic.

Visual

Fare Distribution Chart (See attached)

Insight 3: Patterns of passenger count.

Method

Combined trips by number of passengers, and displayed frequencies.

Finding

The majority of trips had a limited number of passengers.

Interpretation

This means that most taxi rides are not taken by a larger group.

Visual

(Include the number of passengers chart)

5. Reflection and Future Work.

5.1 Technical Challenges

Several technical challenges were encountered in the project:

- Importing big amounts of data into MySQL
- Configuring MySQL LOCAL INFILE
- Working with Flask and React integration.API integration: React and Flask
- Handling Cross-Origin Resource Sharing (CORS)
- Optimizing dashboard performance

5.2 Team Challenges

The team encountered problems with coordination:

- Combining front and back-end aspects.
- Using GitHub for source code management
- Assigning tasks to the team members

These were resolved through regular communication and version control processes.

5.3 Future Work

The following improvements would be made if the system were to be used as a production system:

- Interactive geographic maps
- Advanced filtering and search functions.
- User authentication
- The demand forecast relies on machine learning. Machine Learning demand prediction.
- Cloud deployment
- Real-time transportation monitoring

5.4 Conclusion

The NYC Mobility Dashboard is a great example of how a full stack of technologies can be used to make sense out of transportation-related data. By combining React, Flask, MySQL, and Chart.js, the system offers a dynamic and interactive interface for visualizing urban mobility patterns and transportation trends.